

implementation. To take best advantage of this, OpenSSL provides an engine interface to hook an HSM for hardware acceleration for crypto operations but still use OpenSSL for the TLS protocol. The caller code for OpenSSL's interface will remain the same, with or without hardware acceleration, as the EVP/SSL interface is not impacted.

Also, though the engine interface is targeting primarily HSMs, it need not be hardware only. An external software implementation can also be hooked into OpenSSL using the engine interface, based on the implementation needs.

Overriding the default communication channel in OpenSSL

Contrary to the assumption in the default implementation, a TCP connection may not always be the medium to communicate TLS records. Sometimes, even if the default software crypto implementation of OpenSSL is good enough, a direct connection over TCP may not be possible. For example, in embedded systems, the communication stack may not be directly available. However, a TLS session would have to be established. The TLS records may be exchanged with higher layers over RPC or event notifications, and the higher layers will be involved in establishing a TCP/application layer connection with the server side. This would mean overriding the communication behaviour of OpenSSL. For such an activity, OpenSSL provides an interface called the BIO interface.

The architecture of OpenSSL

As mentioned earlier, OpenSSL can be used as a command line utility or can be used as a library linked into the user's application. Hence, the layering is as described in Figure 2. Below the command line interface are the EVP layer and the SSL interface layer. These are typically the interfaces for the caller code to use the cryptographic functions of OpenSSL. Below that is the actual SSL/TLS default implementation and interfaces to hook in external overrides.

Figure 2 illustrates the layering and architecture of OpenSSL.

Going under the hood

In this section, we go into the open source code and see how OpenSSL is implemented by default. To follow this section,

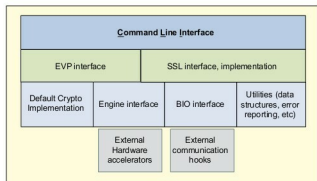


Figure 2: Different layers of SSL

please download the OpenSSL sources tarball from <http://www.openssl.org/source/> and extract it. At the time of writing this article, OpenSSL 1.0.1g is the latest version. So, all illustrations will be based on this version. Remember that since OpenSSL is an evolving toolkit, the file names, variable names, etc, might have changed if you download a different version. So, it is best to download the same version to avoid confusion. For convenience, we will refer to the directory into which OpenSSL has been extracted as $\$(ROOT)$.

Using the engine interface

The engine interface is declared in the header in $\$(ROOT)/include/openssl/engine.h$. There is support for specific hardware in $\$(ROOT)/crypto/engine/$ directory. Each engine is designated an engine ID string, and OpenSSL ships with implementations for some software and interfaces for a few hardware engines, by default. A call to function `ENGINE_by_id` will return the corresponding engine object. For example, 'cswift' is used for CryptoSwift acceleration hardware, while 'ubsec' is for Broadcom uBsec acceleration hardware. The default engine ID is 'openssl', which uses the built-in default software implementation. Once the engine object is got, we need to call `ENGINE_set_default` to use the specified engine. It provides several functions to set external implementations for specific algorithms.

To hook-in an external RSA implementation, write the following code:

```

int ENGINE_set_RSA(ENGINE *e, const RSA_METHOD *rsa_meth);
To hook-in an external DSA implementation:
int ENGINE_set_DSA(ENGINE *e, const DSA_METHOD *dsa_meth);
Likewise, other algorithms:
int ENGINE_set_ECDSA(ENGINE *e, const ECDSA_METHOD *ecdsa_meth);
int ENGINE_set_ECDSA(ENGINE *e, const ECDSA_METHOD *ecdsa_meth);
int ENGINE_set_DH(ENGINE *e, const DH_METHOD *dh_meth);
int ENGINE_set_RAND(ENGINE *e, const RAND_METHOD *rand_meth);
int ENGINE_set_STORE(ENGINE *e, const STORE_METHOD *store_meth);
  
```

Any one or more of the cryptographic functions can be overridden. For the other algorithms that are not set, the default implementation will be used.

If you take a look at $\$(ROOT)/crypto/engine/eng_openssl.c$ you can see how the default engine is loaded in OpenSSL code. The code snippet below is taken from

$\$(ROOT)/crypto/engine/eng_openssl.c$.

```

/* The constants used when creating the ENGINE */
static const char *engine_openssl_id = "openssl";
static const char *engine_openssl_name = "Software engine support";
/* This internal function is used by ENGINE_openssl() and possibly by the
 * "dynamic" ENGINE support too */
  
```